

Number Guessing Game Report

Kathryn Sujai

Project Description

This project features a terminal based number guessing game on python. It begins by explaining the game, and then offers three different difficulty levels that determine how many attempts the user has to guess the randomly generated number. The difficulty defaults to medium in case of an user-error in selecting a difficulty. If easy is selected, the user has 10 attempts, medium allows 7 attempts, and hard allows 5 attempts. The game lets the user know if the guess they input is too high or too low in comparison to the random number. At the end of the game, if they win their attempts are displayed, if not the correct answer is displayed. Both outcomes have a prompt that allows the user to replay the game if they wish.

Approach and Logic Used

I formatted it so that the majority of the code falls under the function `number_guessing_game()` and it would be called as the terminal opens. This function includes the handling of: generating the random number, input/validating difficulty of their choosing, input/validating their guess, handling the loop for each guess and returning feedback and attempts left after each guess. There is a if condition that checks the validity of the difficulty that the user inputs, it defaults to medium after 2 failed attempts from the user. Then, there is a while loop that runs as long as the user's attempts are lower than their attempt limits that is based on difficulty level. I used the try/except method to handle validating user guesses and making sure the input is a valid integer. Once that passes, it is checked with the if condition that actually processes the guess and provides the necessary feedback to the user. After the loop is done, outside of the function there is a while loop that runs if the user selects yes when asked to replay.

Data Structures Used

I used a dictionary in the beginning to store the attempt limit for the difficulty level. This allowed me to have a cleaner code when handling the limit, instead of creating the same loop for each difficulty, it was done in one line.

I also used loops throughout the code for numerous different functional uses. The main loop features the guessing logic that runs until the user guesses correctly or runs off attempts. The replay loop at the end of function allows the user to play multiple rounds while using a simple condition based on user input.

Challenges Faced

I originally struggled with the structure of the difficulty input because if the user entered an invalid answer twice, it would crash the game. Writing several loops inside of each other

would create messy and unneeded code so I made the second inner loop default the difficulty to medium to reduce redundancy and have cleaner code. Also, the dictionary logic made the code cleaner as well.

I also struggled with the replay logic because if the user entered the wrong type of yes, it would also crash. The way I fixed it originally was not right because I used “while replay == "Yes" or "yes" or "YES":”. While logically it makes sense, it was not checking each condition with replay, so I changed it to “while replay == "Yes" or replay == "yes":”. This change cleaned up my code and allowed the replay loop to run smoothly.

Possible Improvements

While I made as many adjustments as I can, I think I could reduce code in a lot of places and make it more concise so that it is easier to understand. The structure of the code is a little difficult to follow. I also think I could update the replay logic to simplify it and clean the code up a little bit. Overall, I think I could add more advancements/enhancements to make the game more interesting to play. This could include feedback that suggests if the user is close or far from the randomly generated number. It would also be interesting if there was a way to track a user’s best attempt.